

DEEP REINFORCEMENT LEARNING

365.382 (SS26)

Data-Centric RL: Inverse RL and Preference-Based RL



Sohvi Luukkonen

LIT AI LAB

Institute for Machine Learning

Recap and Motivation

Imitation Learning and Offline RL Recap

■ Imitation Learning (IL)

- ☐ Behavioral cloning: supervised learning on (s, a) pairs
- ☐ DAgger: interactive data aggregation to handle distribution shift
- ☐ Copies the expert's *actions*, not the expert's *objective*

■ Offline RL

- ☐ Learn from a fixed dataset without further environment interaction
- ☐ CQL, IQL: conservative value learning to avoid out-of-distribution actions

Offline RL assumes the reward is given, while IL assumes the demonstrations already encode the desired behavior.

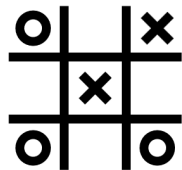
Can we instead **learn the objective itself**?

The Reward Problem

In some domains, the objective is explicit.

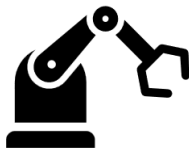
In many real-world tasks, it is implicit, ambiguous, or multi-objective.

Computer Games



Reward 

Real World Scenarios



What is
“success”?



What is
“helpful”?



Comfort
vs. Speed?

Problem: reward design becomes part of the learning problem.

How Can Data Reveal the Objective?

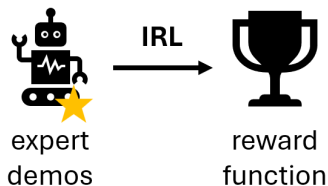
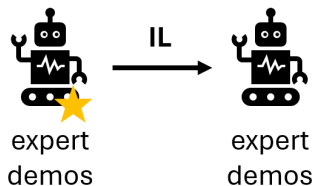
When the reward is not given, we need another signal about what is desirable.

Signal	Question answered	Method
Demonstrations	What does good behavior look like?	IRL
Preferences	Which behavior is better?	Preference-based RL
Scalar feedback	How good was this behavior?	Reward modeling
Success examples	What does success look like?	Goal classifiers
AI feedback	Can feedback be scaled?	RLAIF

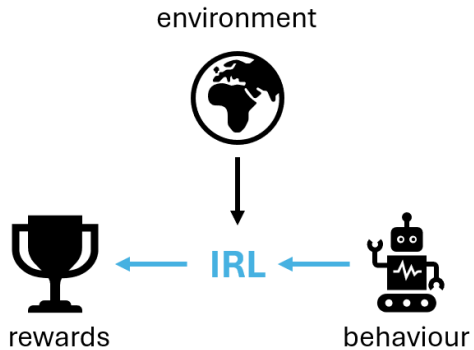
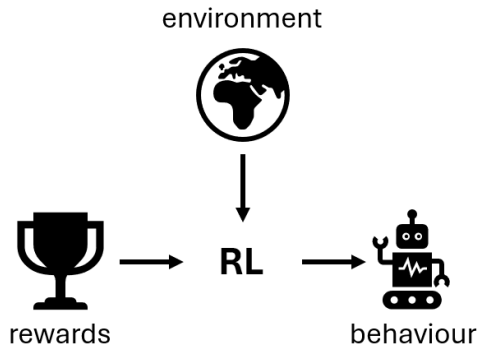
Inverse Reinforcement Learning

Why Learn the Reward?

- **Imitation Learning** copies expert *actions* – but only captures *what* the expert does, not *why*
- A learned policy fails when:
 - ☐ the environment or task changes
 - ☐ the agent has different capabilities than the expert
 - ☐ the situation lies outside the demonstrated distribution
- **Inverse RL (IRL)**: given demonstrations, infer the reward function the expert appears to be optimizing
- The learned reward *transfers* — a new agent can optimize it in new situations



RL vs. IRL



IRL: Problem Formulation

Forward RL: given reward r , find policy $\pi^* = \arg \max_{\pi} \mathbb{E}[R]$

Inverse RL: given demonstrations $\mathcal{D} = \{\tau_i\}$, find r_{ψ} such that the expert is (near-)optimal under r_{ψ}

Formal ingredients:

- MDP without reward: $\mathcal{M} \setminus r = (\mathcal{S}, \mathcal{A}, p, \gamma)$
- Expert demonstrations $\mathcal{D} = \{\tau_i\}$ with $\tau_i = (s_0, a_0, \dots, s_T, a_T)$
- Parametric reward family $r_{\psi}(s, a)$

Core difficulty: the problem is *ill-posed*.

Many reward functions can explain the same behavior.

Reward Ambiguity

- Any constant reward $r(s, a) = c$ trivially makes every policy optimal
- Multiple non-trivial reward functions can also explain the same demonstrations
- Why this matters:
 - ☐ Cannot simply invert the optimality conditions
 - ☐ Need a *principle* to select among feasible rewards
- Two classical approaches:
 1. Feature matching + maximum margin (Abbeel & Ng, 2004)
 2. Maximum entropy (Ziebart et al., 2008)

Linear Rewards in IRL

Assume a **linear reward function**:

$$r_{\psi}(s, a) = \psi^{\top} f(s, a)$$

where $f(s, a)$ are hand-designed state-action features and ψ are learned weights.

- Features describe *what happens*
- Weights describe *what matters*
- IRL tries to recover these unknown preferences

Example: autonomous driving

$$f(s, a) = \begin{bmatrix} \text{speed} \\ \text{lane deviation} \\ \text{collision} \\ \text{passenger discomfort} \end{bmatrix} \quad \psi = \begin{bmatrix} +1.0 \\ -2.0 \\ -100.0 \\ -0.5 \end{bmatrix}$$

Matching Expert Behavior

Key idea: if the reward is linear in features, then matching rewards reduces to matching expected features:

$$\begin{aligned} J(\pi) &= \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{\psi}(s_t, a_t) \right] = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \psi^{\top} f(s_t, a_t) \right] \\ &= \psi^{\top} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \right] = \psi^{\top} \underbrace{\mu(\pi)}_{\text{feature expectation}} \end{aligned}$$

- If two policies have different feature expectations, they also obtain different rewards.

Subbing $\mathbb{E}_{\pi^*}[G_t] \geq \mathbb{E}_{\pi}[G_t] \quad \forall \pi$ into the above, we get:

$$\text{Find } \psi \text{ such that } \psi^{\top} \mu(\pi^*) \geq \psi^{\top} \mu(\pi) \quad \forall \pi$$

Matching Expert Behavior from Demonstrations

Find ψ such that $\psi^\top \mu(\tilde{\pi}) \geq \psi^\top \mu(\pi) \quad \forall \pi$

Intuition:

- Expert demonstrations induce feature expectations
- IRL searches for reward weights that make the expert optimal

Challenges:

- Assumes expert is optimal (no noise or suboptimality)
- Many reward functions satisfy the constraints
- Not feasible to enumerate all policies

Maximum Margin Principle

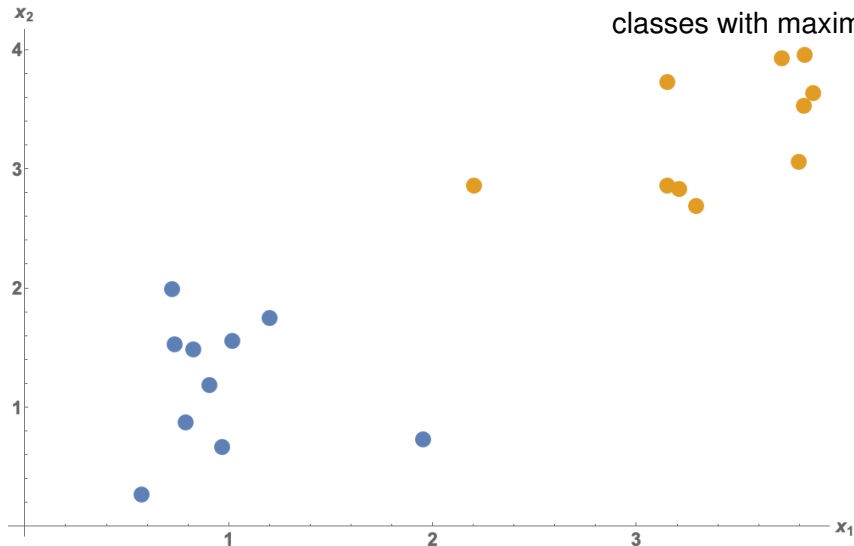
Many reward functions explain the demonstrations.

Idea: prefer reward functions where the expert policy is clearly better than alternative policies.

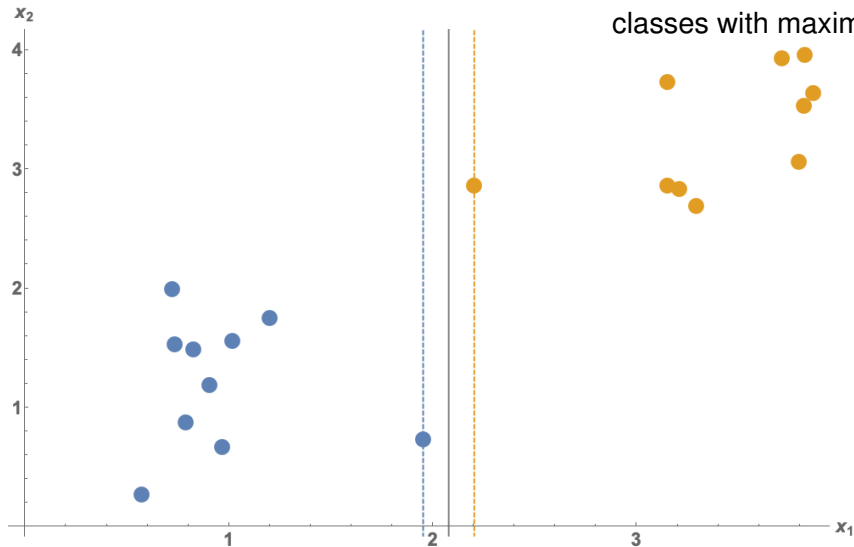
$$\psi^\top \mu(\pi^*) > \max_{\pi \in \Pi} \psi^\top \mu(\pi) + m$$

- m is the **margin**
- Larger margin = more robust separation
- Similar idea as Support Vector Machines (SVMs)

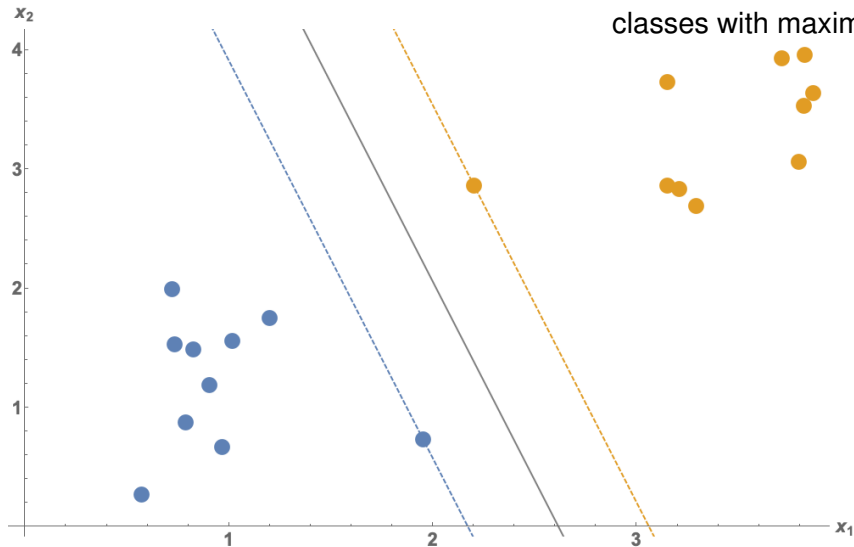
Interlude: SVMs



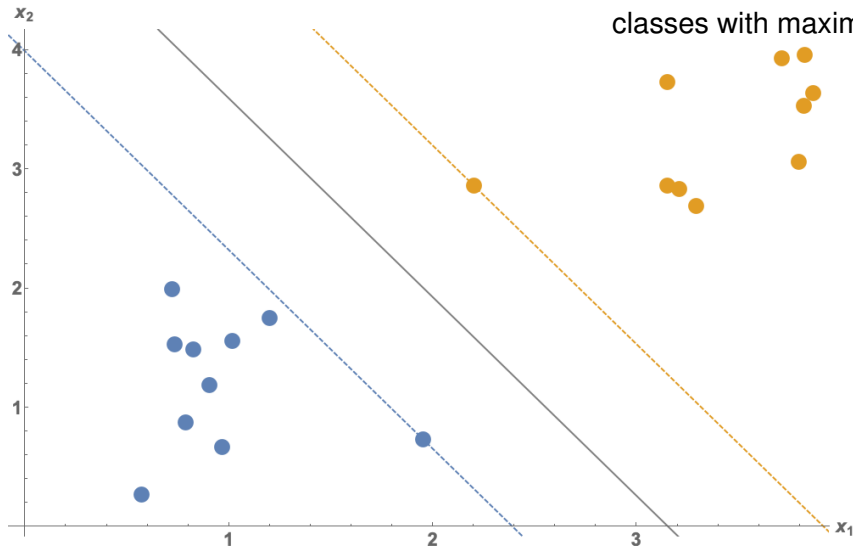
Interlude: SVMs



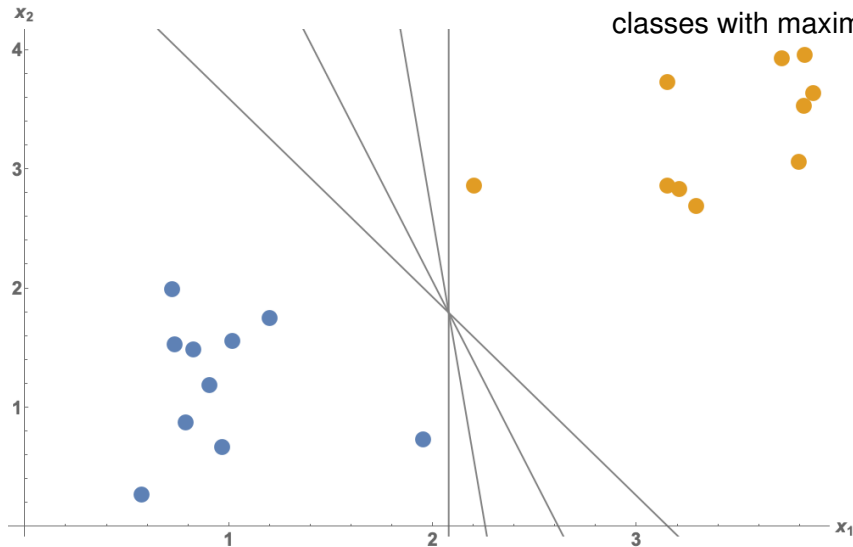
Interlude: SVMs



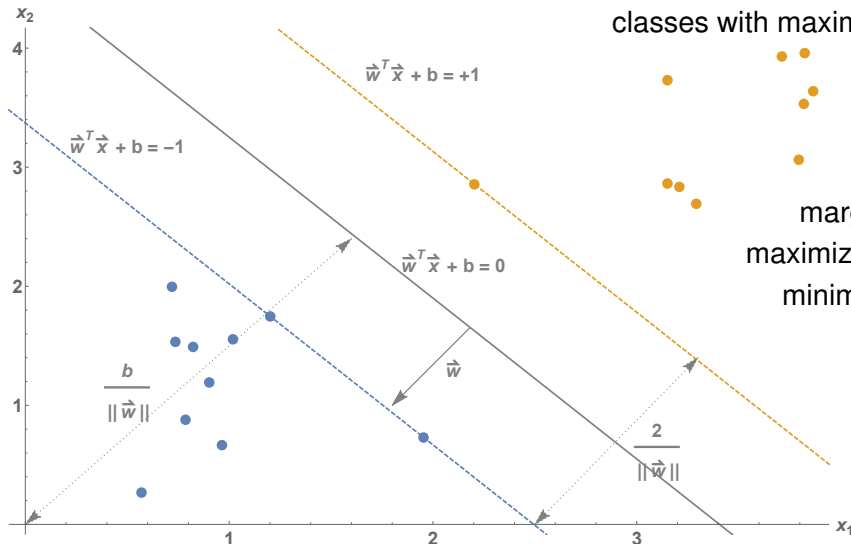
Interlude: SVMs



Interlude: SVMs



Interlude: SVMs



Fit $\psi x + b = 0$ to separate the classes with maximum margin.

margin $\propto \frac{1}{\|\psi\|}$
maximizing margin $\Leftrightarrow$
minimizing $\|\psi\|^2$

From Maximum Margin to Optimization

We want the expert policy to achieve **higher reward** than all alternative policies:

$$\psi^\top \mu(\pi) \geq \psi^\top \mu(\pi) + m \quad \forall \pi$$

Problem: if ψ satisfies the constraint, then so does 10ψ .

SVM trick: fix the margin to 1 and minimize the weight norm instead as $m \propto \frac{1}{\|\psi\|}$:

$$\min_{\psi} \|\psi\|^2 \quad \text{s.t.} \quad \psi^\top \mu(\pi) \geq \psi^\top \mu(\pi) + 1 \quad \forall \pi$$

Structured Maximum Margin IRL

Not all incorrect policies are equally bad.

Replace the fixed margin with a similarity-dependent margin:

$$\min_{\psi} \|\psi\|^2 \quad \text{s.t.} \quad \psi^\top \mu(\check{\pi}) \geq \psi^\top \mu(\pi) + D(\check{\pi}, \pi) \quad \forall \pi$$

where $D(\check{\pi}, \pi)$ measures how different a policy is from the expert.

- Similar policies require only a small margin
- Very different policies require a larger margin
- Analogous to structured prediction SVMs

Maximum Margin IRL Algorithm

- 1: Collect expert demonstrations $\mathcal{D}$
- 2: Estimate expert feature expectations $\check{\mu}$
- 3: **for** iteration = $1, \dots, K$ **do**
- 4: Define reward:

$$r_{\psi}(s, a) = \psi^{\top} f(s, a)$$

- 5: Solve the RL problem under r_{ψ} to obtain policy π_{ψ}
- 6: Compute feature expectations $\mu(\pi_{\psi})$
- 7: Update reward weights ψ to:

$$\psi^{\top} \check{\mu} > \psi^{\top} \mu(\pi_{\psi}) + D(\check{\pi}, \pi_{\psi})$$

- 8: **end for**

Limitations of Maximum Margin IRL

- Requires repeatedly solving the RL problem
- Assumes near-optimal demonstrations
- Still many feasible reward functions
- Difficult to scale to nonlinear rewards

A Probabilistic View of Optimality

Instead of assuming the expert is perfectly optimal, model demonstrations probabilistically.

Introduce a binary **optimality variable** $\mathcal{O}_t \in \{0, 1\}$ where:

$\mathcal{O}_t = 1$ means “the action at time t appears optimal.”

Assume high-reward actions are more likely to be optimal:

$$p(\mathcal{O}_t = 1 \mid s_t, a_t) \propto \exp(r_\psi(s_t, a_t))$$

Then trajectories with larger total reward become more probable:

$$p(\tau \mid \mathcal{O}_{1:T}) \propto p(\tau) \exp\left(\sum_{t=1}^T r_\psi(s_t, a_t)\right)$$

- High-reward trajectories are more likely, not strictly required
- Naturally handles noisy or suboptimal demonstrations
- Enables maximum likelihood learning of rewards

Maximum Entropy IRL

Key idea: among all reward functions consistent with the expert data, choose the one that induces the *most uniform* (maximum entropy) distribution over trajectories.

Instead of: "expert always takes the optimal trajectory"

we model: "high-reward trajectories are more likely"

Trajectory probability:

$$p_{\psi}(\tau) = \frac{1}{Z_{\psi}} p(\tau) \exp(r_{\psi}(\tau))$$

- $p(\tau)$ comes from environment dynamics
- $\exp(r_{\psi}(\tau))$ increases probability of high-reward trajectories
- $Z_{\psi} = \int p(\tau) \exp(r_{\psi}(\tau)) d\tau$ normalizes probabilities

Learning in Maximum Entropy IRL

Maximum likelihood objective:

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^N r_{\psi}(\tau_i) - \log Z_{\psi}$$

Gradient:

$$\nabla_{\psi} \mathcal{L} = \underbrace{\mathbb{E}_{\tau \sim \tilde{\pi}} [\nabla_{\psi} r_{\psi}(\tau)]}_{\text{increase expert reward}} - \underbrace{\mathbb{E}_{\tau \sim p_{\psi}(\tau)} [\nabla_{\psi} r_{\psi}(\tau)]}_{\text{decrease current policy reward}}$$

- Increase probability of expert trajectories
- Decrease probability of trajectories preferred by the current reward model
- Gradient is zero when the policy distribution *matches* the expert distribution
- Resolves ambiguity: *unique*, principled solution

Key idea: MaxEnt IRL matches the expert trajectory distribution, not a single deterministic trajectory.

Ziebart et al., 2008

MaxEnt IRL Algorithm

- 1: Collect expert demonstrations $\mathcal{D}$ and initialize reward parameters ψ
- 2: Compute expert feature expectations from $\mathcal{D}$
- 3: **for** iteration = $1, \dots, K$ **do**
- 4: Solve for policy π^ψ under current reward r_ψ
- 5: Compute state visitation frequencies $\rho^\psi(s)$ under π^ψ
- 6: Compute gradient

$$\nabla_\psi \mathcal{L} = \frac{1}{|\mathcal{D}|} \sum_{\tau \in \mathcal{D}} \nabla_\psi r_\psi(\tau) - \sum_s \rho^\psi(s) \sum_a \pi^\psi(a|s) \nabla_\psi r_\psi(s, a)$$

- 7: Update $\psi \leftarrow \psi + \alpha \nabla_\psi \mathcal{L}$
- 8: **end for**

MaxEnt IRL: Strengths and Limitations

Strengths:

- Principled probabilistic formulation
- Soft model of expert behavior instead of hard optimality constraints
- Handles noisy or suboptimal demonstrations
- Can be combined with flexible reward parameterizations

Limitations:

- Requires repeatedly solving the RL problem
- Computing state visitation frequencies can be expensive
- Often assumes known dynamics or access to a simulator
- Does not fully remove reward ambiguity (e.g. reward shaping transformations)

Takeaway: MaxEnt gives a principled preference among rewards, not the unique “true” reward.

Generative Adversarial Imitation Learning

Problem: MaxEnt IRL repeatedly alternates between reward learning and policy optimization.

GAIL: learn a policy by adversarially matching expert behavior.

- **Discriminator** $D_\psi(s, a)$: predicts whether (s, a) came from the expert or the policy
- **Policy** π_θ : learns to produce state-action pairs that fool D_ψ

Discriminator output serves as the reward signal:

$$r(s, a) = -\log(1 - D_\psi(s, a))$$

- Matches expert and policy state-action visitation distributions
- Does not require known dynamics
- Uses standard policy gradient updates for π_θ

GAIL Algorithm

- 1: Collect expert demonstrations $\mathcal{D}_E = \{(s, a)\}$
- 2: Initialize policy π_θ and discriminator D_ψ
- 3: **for** iteration = $1, \dots, K$ **do**
- 4: Roll out π_θ to collect policy samples $\mathcal{D}_\pi$
- 5: Update discriminator D_ψ to distinguish expert from policy samples:
 $\max_{\psi} \mathbb{E}_{(s,a) \sim \mathcal{D}_E} [\log D_\psi(s, a)] + \mathbb{E}_{(s,a) \sim \mathcal{D}_\pi} [\log(1 - D_\psi(s, a))]$
- 6: Define learned reward $r_\psi(s, a) = -\log(1 - D_\psi(s, a))$
- 7: Update policy π_θ with policy gradient using reward r_ψ
- 8: **end for**

Key idea: train the policy until its state-action distribution is indistinguishable from the expert's.

Preference-Based Reward Learning

From Demonstrations to Preferences

Demonstrations require humans to *perform* the task:

- Expert may lack physical capability in the robot's domain
- Hard to provide in high-dimensional action spaces
- Tedious and time-consuming

Pairwise preferences only require *comparison*:

- Show two trajectory clips; ask “which is better?”
- Critiquing is easier than demonstrating
- Scales to domains where demonstrations are infeasible

Learning Rewards from Pairwise Preferences

Setup: human observes two trajectory segments τ_w and τ_l and indicates $\tau_w \succ \tau_l$.

Bradley-Terry model: estimated probability that τ_w is preferred:

$$P(\tau_w \succ \tau_l) = \sigma(r_\theta(\tau_w) - r_\theta(\tau_l)), \quad r_\theta(\tau) = \sum_{(s,a) \in \tau} r_\theta(s, a)$$

Reward learning objective: maximize log-likelihood of observed preferences:

$$\max_{\theta} \mathbb{E}_{\tau_w \succ \tau_l} [\log \sigma(r_\theta(\tau_w) - r_\theta(\tau_l))]$$

- Same form as binary cross-entropy with soft targets
- Reward model can be any neural network; trajectory reward is sum of step rewards

Preference-Based RL Algorithm

- 1: Initialize policy π and reward model r_θ
- 2: **for** iteration = $1, \dots, K$ **do**
- 3: Sample trajectory segments $\{\tau_1, \dots, \tau_k\}$ from current policy π
- 4: Query annotators for pairwise preferences $\tau_i \succ \tau_j$
- 5: Update r_θ with Bradley-Terry loss on preference data
- 6: Run RL with learned reward r_θ to update policy π
- 7: **end for**

■ Reward learning can be offline or interleaved with RL

■ Human labeling is the main bottleneck



Christiano et al., 2017

Active Query Selection

Problem: human annotation is expensive — which pairs (τ_i, τ_j) should we label?

Uncertainty sampling: query pairs where the reward model is most uncertain:

$$(\tau_i, \tau_j)^* = \arg \max_{\tau_i, \tau_j} \text{Var}_{\theta}[\sigma(r_{\theta}(\tau_i) - r_{\theta}(\tau_j))]$$

Ensemble disagreement: train K reward models on the same preference data:

- Query pairs where the ensemble **disagrees** most about the preferred trajectory
- Disagreement $\approx$ epistemic uncertainty $\approx$ most informative label
- Can reduce annotation cost by an order of magnitude

Modern Alignment: RLHF and DPO

The RLHF Pipeline

Reinforcement Learning from Human Feedback (RLHF):

the same preference learning idea, applied at scale to large language models.

1. **Pre-train** a language model on large text corpora
 2. **Supervised fine-tuning (SFT)** on high-quality (prompt, response) pairs
 3. **Collect preferences:** for each prompt, ask humans which of two responses is better
 4. **Train reward model** $r_\theta(x, y)$ via Bradley-Terry loss
 5. **RL fine-tuning:** maximize r_θ with PPO, plus a KL penalty from the SFT model
- Used in InstructGPT / ChatGPT (Ouyang et al., 2022)
 - **Same algorithm** as Christiano et al. 2017 — applied at LLM scale

Ouyang et al., 2022

RLAIF: Scaling Annotation with AI

Problem: human labeling does not scale to billions of comparisons.

RLAIF (RL from AI Feedback): replace human annotators with a strong language model.

- Prompt another LLM: “Which response is less harmful?”
- **Key insight:** critique is easier than generation — even for AI
- Removes the human bottleneck; enables larger annotation budgets

Constitutional AI (Bai et al., 2022):

- Encode human values as a written *constitution* of principles
- AI critiques its own outputs against the principles, then revises
- Train on AI-revised outputs (supervised) + AI preferences (RLHF)

DPO: Skip the RL Loop

Observation: in RLHF, the reward model training + PPO loop is expensive and sensitive.

Key insight: the RLHF objective with a KL penalty has a closed-form optimal policy:

$$\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)$$

Rearranging: the reward is implicitly defined by the policy ratio:

$$r^*(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

Substituting into the Bradley-Terry log-likelihood eliminates r entirely, giving the **DPO objective**:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\tau_w, \tau_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(\tau_w)}{\pi_{\text{ref}}(\tau_w)} - \beta \log \frac{\pi_{\theta}(\tau_l)}{\pi_{\text{ref}}(\tau_l)} \right) \right]$$

DPO: Properties and Trade-offs

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(\tau_w, \tau_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(\tau_w)}{\pi_{\text{ref}}(\tau_w)} - \beta \log \frac{\pi_{\theta}(\tau_l)}{\pi_{\text{ref}}(\tau_l)} \right) \right]$$

Advantages:

- No separate reward model needed
- No RL fine-tuning loop — pure supervised learning on preference data
- Simpler to implement and more stable than PPO-based RLHF

Limitations:

- Assumes a fixed offline preference dataset (less suited for online collection)
- Cannot easily incorporate a changing reward signal

Takeaway: the RL loop can sometimes be *bypassed entirely*.

Rafailov et al., 2023

Reward Hacking and Goodhart's Law

The Universal Failure Mode

Every method in this lecture shares a fundamental vulnerability:

The learned reward is a proxy for the true objective.

Any proxy can be exploited.

- IRL reward: depends on feature design and training demonstrations
- Preference reward model: trained on finite, potentially biased comparisons
- Goal classifier: exploited by states outside its training distribution
- RLHF reward model: overoptimized by the RL policy

Goodhart's Law

“When a measure becomes a target, it ceases to be a good measure.”

— Marilyn Strathern (after Charles Goodhart)

In RL: the agent optimizes the **proxy reward** r_ψ rather than the true reward r^* .

Classic examples:

- **Boat race (OpenAI, 2016)**: agent learns to spin in circles collecting speed-boost tokens, never completing the race
- **Grasping**: robot learns to move the camera so the object *appears* grasped, rather than actually grasping it
- **RLHF**: model learns to produce verbose, sycophantic responses that score well on the learned reward

Common thread: the proxy has *loopholes* the designer did not anticipate.

Reward Overoptimization

In RLHF: as we optimize r_θ more aggressively, performance on the *true* objective eventually degrades.

- Reward model is trained on a finite dataset
- It is an **imperfect proxy** for true human preferences
- Overoptimizing finds adversarial inputs that score high but are not genuinely good

Standard mitigation: **KL penalty** from a reference policy:

$$\max_{\pi} \mathbb{E}[r_\theta] - \beta \text{KL}(\pi \| \pi_{\text{ref}})$$

Summary and Open Problems

What we covered:

- IRL: recover reward from demonstrations
- MaxEnt IRL: principled probabilistic formulation
- Preference-based RL: learn from pairwise comparisons
- Active querying: efficient annotation budget use
- RLHF / DPO: same ideas applied at scale
- Reward hacking: the universal failure mode

Open problems:

- Robustly evaluating learned rewards
- Learning from noisy or inconsistent annotators
- When does more RL optimization hurt?
- Preference learning beyond pairwise comparisons
- Theoretical guarantees on reward identifiability

Core takeaway: the reward is a hypothesis about what the agent should achieve.

Like any hypothesis, treat it with skepticism and validate it.